# AI Engineering Internship Assignment - Tri9T AI

## Key Terms (if any of this is new to you, that's fine — quick primer)

- **Test case**: a specific, repeatable check with steps and an expected result — e.g. "Simulate cuff pressure exceeding safe limit → device should display E3 and auto-deflate within 2 seconds." it should be concrete enough that someone else could execute it.
- **QA (Quality Assurance)**: the practice of systematically checking that a product behaves as specified, especially important for medical devices where a missed bug can mean patient harm.
- **Traceability**: being able to point from a test case back to the exact requirement/section it came from, so you can prove every requirement is actually covered.
- **Stale traceability**: when a requirement's text changes after a test case was generated from it, and nobody flags that the test case might no longer be accurate. This is a real, common failure mode in regulated software and it's the hardest part of this assignment.

## Input Format

**Input Data:** AI Internship Assignment Files-CT-200

You will work with the **CT-200 Manual (PDF)** provided in the `data/` folder of the **AI Internship Assignment Files – CT-200** package.

Treat this as you would a real engineering or regulatory document—not as a clean tutorial dataset. Before writing any extraction logic, carefully inspect the document and understand its structure. Do not assume the formatting is perfectly consistent just because the first few pages follow a pattern.

Your task is to build an **OCR-based document extraction pipeline** that reconstructs the document hierarchy. We expect you to use appropriate OCR and document understanding techniques to identify and preserve:

- Document titles and headings
- Section and subsection hierarchy
- Paragraphs and lists
- Tables (where possible)
- Figures/images and their captions (if applicable)
- The parent-child relationships between sections

The goal is **not** to build a generic parser for every PDF. Instead, build a solution that correctly handles the structural variations present in this specific CT-200 document without silently losing, merging, or mis-parenting content. A parser that produces a clean-looking but incorrect hierarchy is considered worse than one that clearly exposes unsupported cases.

In your **Approach Document**, describe:

- The OCR/document parsing approach you selected and why.
- The hierarchy reconstruction strategy you used.
- The structural inconsistencies or edge cases you discovered in the PDF.
- What your initial implementation failed to handle.
- How you identified those failures (manual inspection, validation scripts, visual comparison, test cases, etc.).
- What changes you made to improve the extraction quality.

We are interested not only in your final output, but also in your engineering process, debugging methodology, and how you validated the correctness of the extracted document hierarchy.

# The Problem

You're given a technical document describing a fictional medical device, the **CardioTrack CT-200 Home Blood Pressure Monitor**. Build a backend API (language/framework of your choice) that turns this document into a browsable, structured, *versioned* tree, and generates QA test-case ideas from sections a user selects — while keeping traceability valid as the document changes.

## What you need to build

1. **Ingestion & structuring** Parse the pdf into a hierarchical tree and persist it. Each node retains heading, level, body text, parent/child relationships, and a **content hash** (used later for staleness detection). Your parser must correctly handle every irregularity listed above — write at least 3 explicit unit tests targeting them (e.g. a test asserting the duplicate-heading case produces two distinct node IDs with correct parents).

2. **Document versioning** Support re-ingesting a modified version of the manual (`data/ct200_manual_v2.pdf`, also provided) as a *new version* of the same document, without destroying version 1. Nodes that are semantically unchanged between v1 and v2 should be recognized as the same logical node (not duplicated); nodes whose body text changed should be flagged. You choose the matching strategy (path-based, hash-based, fuzzy title match, etc.) — justify it in your approach doc, including where it breaks.

3. **Browse API**

    - List top-level sections (with a version parameter/default to latest)
    - Get a specific node by ID, including children, full text, and its content hash
    - Search/filter across node headings or text
    - Given a node ID, return whether it has changed across versions, and if so, a lightweight diff summary
4. **Selection API** Submit a set of node IDs as a named "selection." Selections must be **version-pinned** — i.e., a selection references specific node+version pairs, so that if the document is later re-ingested, old selections still resolve to the exact text they were created against.

5. **LLM-powered generation API** Given a selection, reconstruct the relevant text, send it to an LLM with a prompt you design, and generate 3–5 QA test case ideas. Your system will be run against real LLM output, which means at some point it will get back a response that's malformed, incomplete, or not what your prompt asked for — decide how your system behaves when that happens, and why. "It usually works" is not a design. Store the generated output linked to the selection *and* to the exact node content it was generated from, in a way that survives the document being re-versioned later (see below). Also decide what happens if the same selection is submitted twice — we're not telling you the policy, but we do expect you to have one and be able to defend it.

6. **Staleness / impact detection** A generation (a set of test cases) was created from specific document text. That text can later change when the document is re-ingested. Design a way for your system to tell a user, at retrieval time, whether a previously generated test case still reflects the current document — and be honest about the limits of your approach (e.g. does a one-word wording change get treated the same as a changed pressure threshold? should it?).

7. **Retrieval API** Fetch previously generated test cases by selection ID or node ID. Whatever mechanism you built in item 6, it needs to actually be visible here — a correct staleness check nobody can query for is not a finished feature.

## Expected tech stack

- **FastAPI**, **Pydantic**, **SQLAlchemy + SQLite** for the tree, versions, and selections
- A **NoSQL** store (MongoDB local/Atlas free tier, or a well-justified JSON store) for LLM-generated output
- **Git**, used properly — we will look at your commit history, not just the final diff

If you deviate from this stack, justify it in your approach document.

## Other constraints & freedoms

- Any LLM provider is fine (free-tier Groq/Gemini/OpenRouter etc.); you will be penalized for skipping structured-output validation, not for which provider you picked.
- You may extend the sample documents, but the parsing/versioning/staleness logic is the point — don't spend your time authoring content.
- No frontend required. A Postman collection, curl examples, or a script hitting your API is sufficient, but it must demonstrate the versioning + staleness flow end-to-end, not just happy-path CRUD.

### Explicitly out of scope (don't over-build)

- Auth/user accounts
- A generic pdf parser for arbitrary documents
- Auto-regeneration of stale test cases
- A UI

# Deliverables

1. **GitHub repository** with real incremental commit history.
2. **README.md** — setup/run instructions, env vars, how to test, and how to trigger the v1 → v2 re-ingestion flow specifically.
3. **Approach document** — data model, tree-parsing decisions (including how you handled each listed irregularity), your version-matching strategy and its known failure modes, LLM prompt design + structured-output/retry strategy, and what you'd do differently with more time.
4. **Submission email** with links to the repo and approach doc.

# Decision log (required, part of your approach doc)

Answer these directly — a sentence or two each is fine, but they need to be *your* reasoning, not a generic best-practice answer:

1. What's the one part of this system most likely to silently give wrong results without erroring? How would you catch it?
2. Where did you choose simplicity over correctness because of time, and what would break first if this went to production as-is?
3. Name one input (to your parser, your versioning matcher, or your LLM call) that you did *not* handle, and what your system does when it sees it.

# Note

A submission that works end-to-end but hand-waves the decision log will score lower than one that's rougher around the edges but shows real judgment in it. What matters most is that you understand and can defend every part of what you submit — including the parts you know are

weak. Be ready to walk through your code, explain a design choice you'd reconsider, and make a live change to it if shortlisted for the next round.